



Módulo 1. Seguridad en redes



1.2 Protección de redes mediante firewalls

Marcos Sánchez-Élez, Inmaculada Pardines – Guadalupe Miñana - Sara Román - Eduardo Huedo

Necesidad de los cortafuegos

- El acceso a Internet permite al mundo exterior interactuar con la red local, lo que supone una amenaza para la organización
 - Es posible equipar cada sistema con funciones avanzadas de seguridad, pero esto puede no ser suficiente o asequible
- Un **cortafuegos complementa** los servicios de **seguridad del sistema** bloqueando el tráfico no permitido a una red o a un sistema
 - Se suele insertar entre la red local e Internet
 - Puede ser un único sistema o un conjunto de sistemas que cooperan para realizar la función de cortafuegos
 - Establece un enlace controlado y un muro externo de seguridad
 - Seguridad perimetral
 - Protege la red local de ataques desde Internet
 - **Proporciona un punto único de choque donde se puede imponer seguridad y auditoría**



Funcionalidad del Cortafuegos

■ Filtrado del tráfico en la red

Objetivos de diseño

- ✓ Todo el tráfico desde dentro hacia fuera, y viceversa, debe pasar a través del cortafuegos
- ✓ Solo debe dejar pasar el tráfico autorizado, definido por una política local de seguridad
- ✓ Debe ser inmune a la intrusión

Capacidades

- ✓ Proteger la red prohibiendo que el tráfico sospechoso entre o salga de la red
- ✓ Registrar la actividad (LOG) para posibles auditorías o alarmas
- ✓ Realizar traducción de direcciones NAT
- ✓ Servir de plataforma para IPsec
 - Se puede usar para implementar una VPN

Limitaciones

No pueden proteger de:

- ✓ Ataques que evaden el cortafuegos
- ✓ Amenazas internas
- ✓ Comunicaciones inalámbricas entre sistemas locales en diferentes lados del cortafuegos
- ✓ Dispositivos portátiles usados e infectados fuera de la red corporativa y luego conectados y usados internamente

Implementaciones de cortafuegos

- Como **software instalado** sobre un computador con un sistema operativo estándar
 - El sistema debe estar muy fortificado, porque los SO añaden muchas funcionalidades que pueden ser una fuente de vulnerabilidades
 - Es necesario:
 - Deshabilitar cuentas de usuario no necesarias y subsistemas no usados
 - Desactivar servicios innecesarios
 - Cerrar los puertos que no se necesiten
 - etc
- Como un aparato **hardware dedicado** (*appliance*)
 - Opción más segura ya que tienen una versión básica de SO (menos susceptible a vulnerabilidades)

Tipos de filtros

Método	Descripción	Ventajas	Desventajas
Filtros de Paquete	Un cortafuegos de filtro de paquete lee cada uno de los datos que viajan a través de una LAN. Puede leer y procesar paquetes según la información de sus cabeceras, y filtrar el paquete basándose en un conjunto de reglas programables implementadas por el administrador del cortafuegos. El kernel de Linux tiene una funcionalidad de filtro de paquetes predefinida, mediante el subsistema del kernel Netfilter.	Personalizable a través del utilitario iptables. No necesita cualquier personalización del lado del cliente, dado que toda la actividad de red se filtra en el nivel del encaminador (router) en vez de a nivel de aplicación. Debido a que los paquetes no se transmiten a través de un proxy, el rendimiento de la red es más rápido debido a la conexión directa entre el cliente y el equipo remoto.	No se pueden filtrar paquetes para contenidos como en los cortafuegos proxy. Procesa los paquetes en la capa del protocolo, pero no los puede filtrar en la capa de una aplicación. Las arquitecturas de red complejas pueden complicar la configuración de las reglas de filtrado de paquetes, especialmente si se lo hace con el enmascarado de IP o con subredes locales y redes de zonas desmilitarizadas.

Tipos de filtros

Método	Descripción	Ventajas	Desventajas
NAT	Network Address Translation(NAT), coloca direcciones IP de subredes privadas, detrás de un pequeño grupo de direcciones IP públicas, enmascarando todas las peticiones hacia un recurso, en lugar de varios. El kernel de Linux tiene una funcionalidad NAT predefinida, mediante el subsistema del kernel Netfilter.	Se puede configurar transparentemente para máquinas en una LAN. La protección de muchas máquinas y servicios detrás de una o más direcciones IP externas simplifica las tareas de administración. La restricción del acceso a usuarios dentro y fuera de la LAN se puede configurar abriendo o cerrando puertos en el cortafuego/puerta de enlace NAT.	No se puede prevenir la actividad maliciosa una vez que los usuarios se conecten a un servicio fuera del cortafuegos.

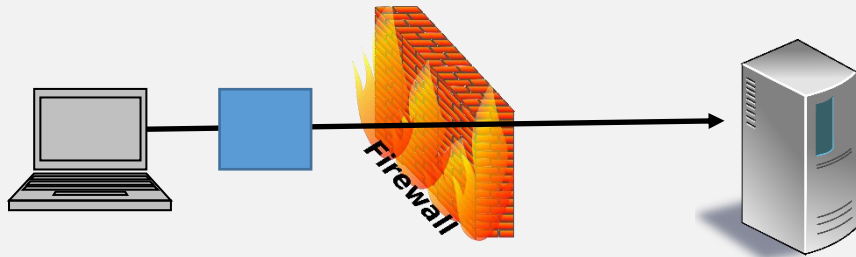
Tipos de filtros

Método	Descripción	Ventajas	Desventajas
Proxy	El cortafuegos proxy filtra todas las peticiones de los clientes LAN de un determinado protocolo, o tipo, hacia una máquina proxy , la que luego realiza esas mismas peticiones a internet, en nombre del cliente local. Una máquina proxy actúa como un búfer entre usuarios remotos maliciosos y la red interna de máquinas clientes.	Los administradores tienen el control sobre qué aplicaciones y protocolos funcionan fuera de la LAN. Algunos servidores proxy, pueden hacer cache de datos accedidos frecuentemente en vez de tener que usar la conexión a Internet para bajarlos. Esto ayuda a reducir el consumo de ancho de banda. Los servicios de proxy pueden ser registrados y monitorizados más de cerca, lo que permite un control más estricto sobre el uso de los recursos de la red.	Los proxies son a menudo específicos a una aplicación (HTTP, Telnet, etc.), o restringidos a un protocolo (la mayoría de los proxies funcionan sólo con servicios que usan conexiones TCP). Los servicios de aplicación no se pueden ejecutar detrás de un proxy, por lo que sus servidores de aplicación deben usar una forma separada de seguridad de red. Los proxies se pueden volver cuellos de botellas, dado que todos los pedidos y transmisiones son pasados a través de una fuente, en vez de hacerlo directamente desde el cliente al servicio remoto

Tipos

Filtrado de paquetes

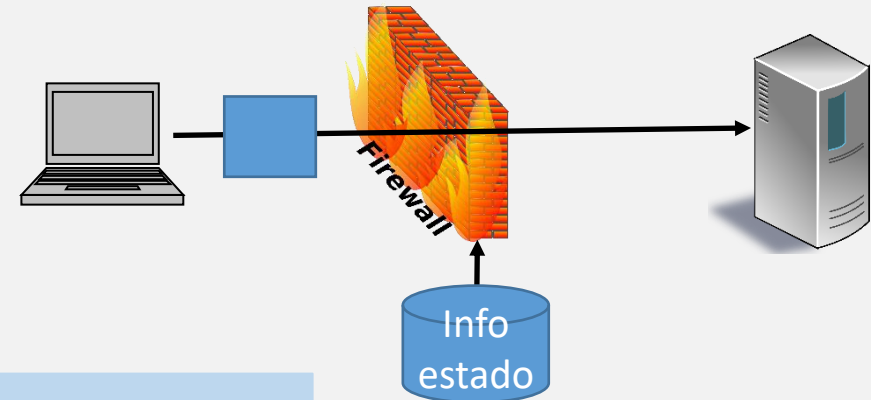
Toman **decisiones** a partir de la **inspección del paquete actual**



Inspeccionan las
cabeceras IP y TCP

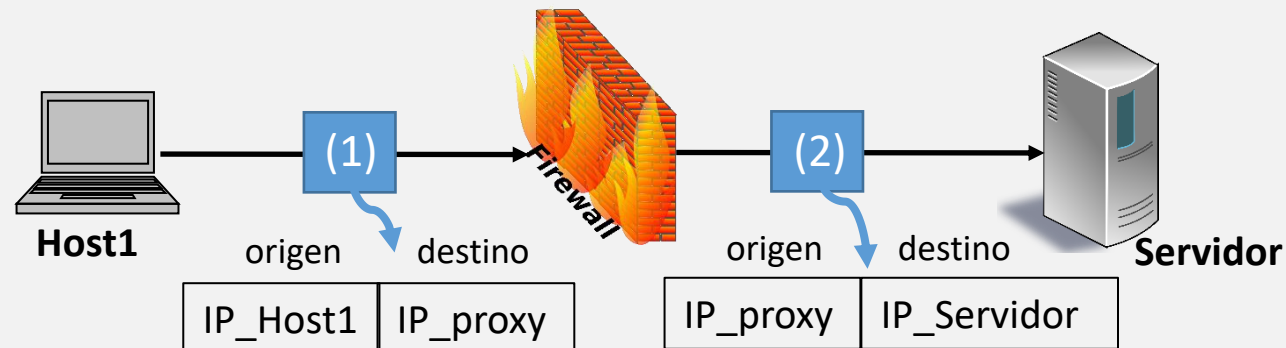
Inspección de estado

Toman **decisiones** a partir de la **inspección del paquete actual** y del **estado** de la **conexión** (paquetes previos pertenecientes a la misma conexión)



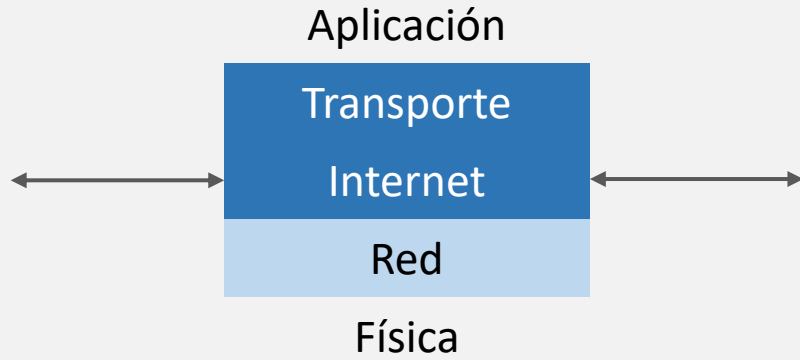
Proxy

Actúa como un **intermediario**
Actúan a nivel capa de **aplicación** o **transporte**



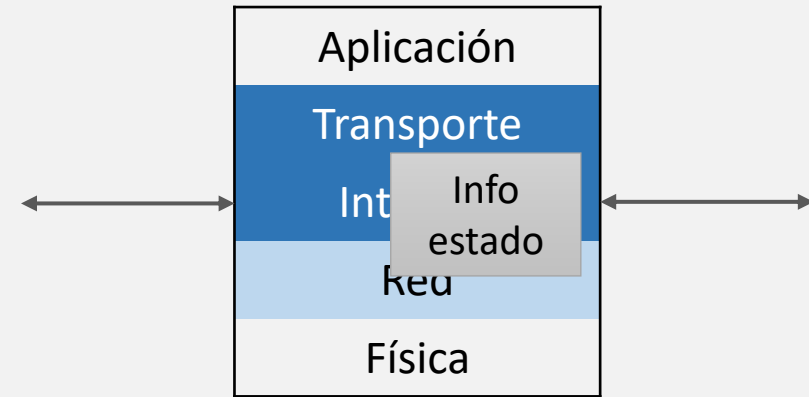
Tipos

Filtrado de paquetes

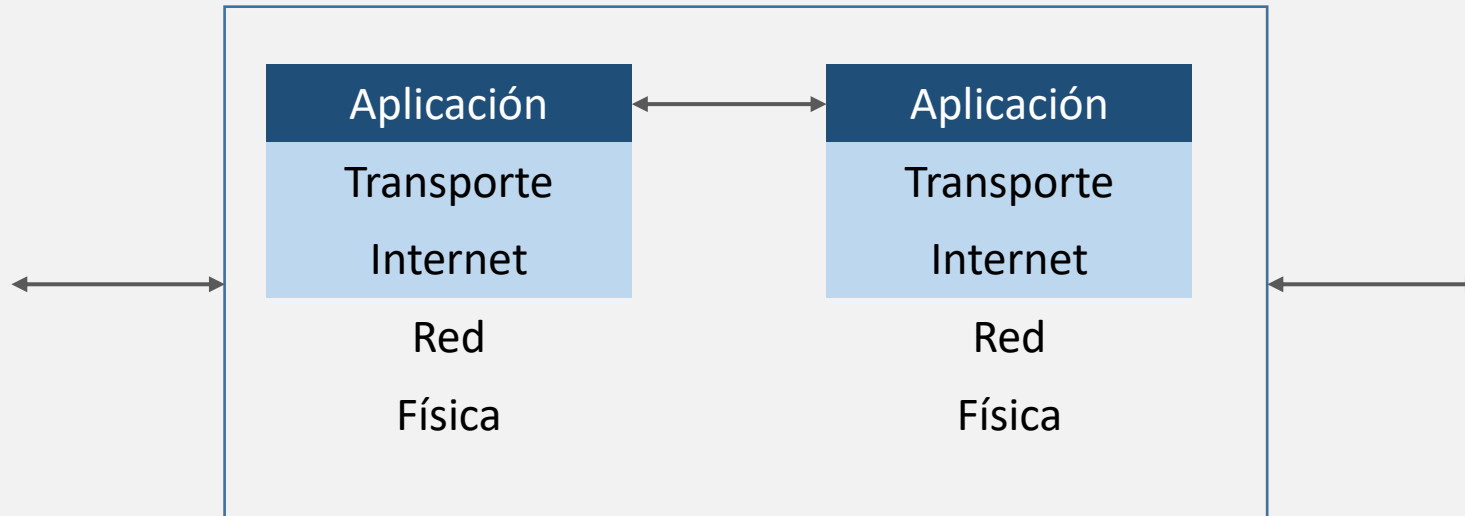


Inspeccionan las
cabeceras IP y TCP

Inspección de estado



Proxy



Cortafuego de filtrado de paquetes

Cortafuegos de filtrado de paquetes

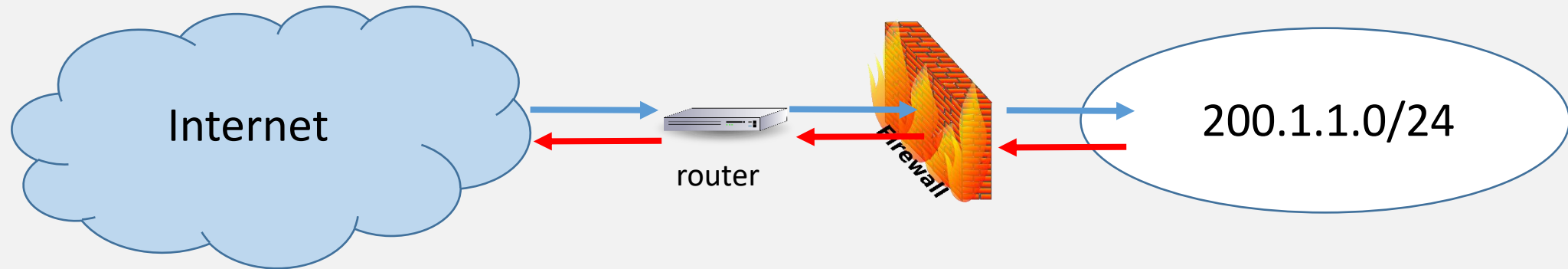
- El filtrado se realiza en la **capa de red / transporte**
- **Inspecciona la cabecera, no el contenido**
 - Sentido de la comunicación (in, out)
 - Dirección IP origen y destino
 - Número de puerto TCP o UDP origen o destino
 - Tipo de protocolo (TCP, UDP, ICMP ...)
 - Flags TCP
 - Tipo de mensaje ICMP
- Es simple, transparente y rápido
- Ejemplo: Netfilter/iptables en Linux

Filtrado de paquetes

- Se configuran como **un conjunto de reglas basadas en coincidencias** con los campos de las **cabeceras** de los protocolos
 - **Las reglas se aplican en orden**
 - Si hay coincidencia en una de las reglas, se ejecuta la acción definida para esa regla, p. e., aceptar o descartar el paquete
 - Si no coincide con ninguna regla, se toma una acción por defecto

Ejemplo de filtrado de paquetes

- Supongamos que desde la red interna están prohibidas las conexiones http con el exterior y están permitidas las conexiones https



Dirección	IP origen	IP destino	Protocolo	Puerto origen	Puerto destino	Acción
Saliente	200.1.1.0/24	ANY	TCP	>1023	80	DENY
Entrante	ANY	200.1.1.0/24	TCP	80	>1023	DENY
Saliente	200.1.1.0/24	ANY	TCP	>1023	443	ACCEPT
Entrante	ANY	200.1.1.0/24	TCP	443	>1023	ACCEPT

Filtrado de paquetes

■ Ventajas

- Escalabilidad
- Independientes de la aplicación
- Alto rendimiento ya que implican poco procesamiento de los paquetes

■ Desventajas

- No puede impedir ataques que exploten vulnerabilidades de las aplicaciones
- No suele soportar esquemas avanzados de autenticación de usuarios
- Funcionalidad de registro (*log*) limitada
- La mayoría no son capaces de detectar IP *spoofing*
- No pueden detectar ataques de fragmentación de paquetes

Se usan como primera línea de defensa y se complementan con firewalls más sofisticados

Cortafuego de filtrado de paquetes por inspección de estado

Cortafuegos de Inspección de estado

- **Controlan el estado de una conexión** recogiendo información de los paquetes que forman parte de la misma y guardándola en una **tabla de estado**
 - Siempre almacenan direcciones IP, números de puerto y estado de la conexión
 - También pueden registrar los números de secuencia TCP para evitar ataques que dependan de ellos (ej. ataque de repetición)
 - También pueden identificar paquetes de nuevas conexiones que están relacionadas con las ya establecidas (ej. transferencias de datos FTP o mensajes ICMP asociados)
- Toman decisiones de forma similar al filtrado de paquetes (chequeando las cabeceras) pero teniendo en cuenta el contenido de la tabla de estado
 - Se puede **permitir tráfico** hacia puertos efímeros solo para **conexiones previamente establecidas** desde esos puertos
 - Nunca se aceptaría un mensaje entrante SYN+ACK si no ha habido un mensaje saliente SYN
 - Un DNS Response (UDP) del exterior solo se aceptaría si ha habido previamente un DNS query

Ejemplo de tabla de estado de conexiones

- **Pasos** que sigue la **configuración** de la **tabla**:

- El dispositivo 192.168.1.100 quiere conectarse con el 192.0.2.71 y le envía un paquete
- Se comprueban las reglas del firewall y, si está permitido, el paquete se envía. Se añade una entrada a la tabla con estado de **conexión iniciada**
- Como se trata de una conexión TCP, concluido el handshake, el estado cambia a **conexión establecida**

IP origen	Puerto origen	IP destino	Puerto destino	Estado de la conexión
192.168.1.100	1030	192.0.2.71	80	Initiated
192.168.1.102	1031	10.12.18.74	80	Established
192.168.1.101	1033	10.66.32.122	25	Established

- Netfilter/**iptables** puede convertirse en un filtro de **inspección de estado** usando el **módulo conntrack** o el **módulo state**
- Pueden sufrir **ataques DoS** al ser bombardeados con información falsa que llena la tabla de estado

Ejemplo de filtrado de paquetes: Netfilter / IPTABLES

¿Qué es Netfilter / iptables?

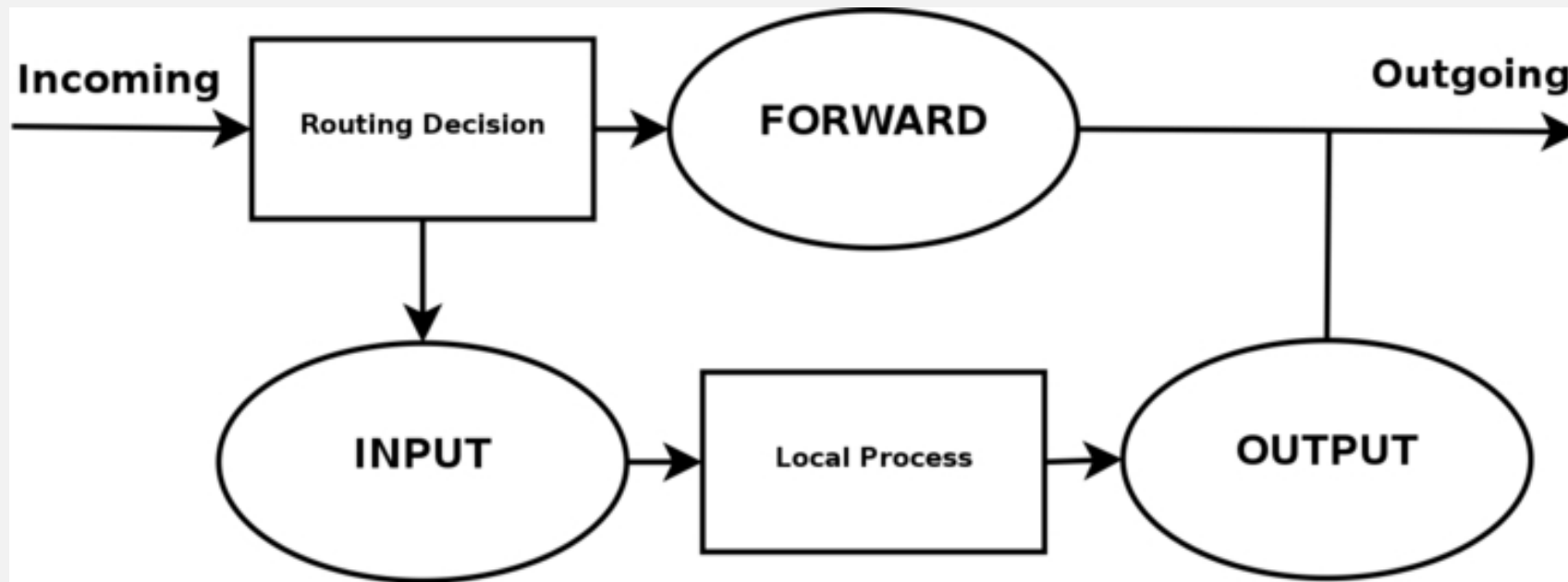
- **Netfilter** es un *framework* disponible en el núcleo de Linux para la interceptación y manipulación de paquetes de red
 - Proporciona filtrado de paquetes, traducción de direcciones [y puertos] (NA[P]T) y manipulación de paquetes
- **iptables** es una herramienta, construida sobre Netfilter, que implementa cortafuegos de filtrado de paquetes
 - Estructura de tabla genérica para la definición de conjuntos de reglas
 - Las reglas se definen en un archivo de texto plano
 - El fichero se ejecuta como administrador del sistema
 - Las reglas se cargan en el núcleo del sistema operativo y se comparan con la cabecera del paquete, ejecutándose una determinada acción cuando haya una coincidencia
 - Ofrece traducción NAT de direcciones de red

Algunas definiciones

- **Tabla:** contiene cadenas predefinidas o de usuario
 - **Tabla filter** para filtrado de paquetes
 - **Tabla nat** para traducción de direcciones y puertos
- **Cadena:** contiene una secuencia de reglas que se van probando consecutivamente
- **Regla:** establece los criterios de coincidencia y especifica un objetivo (o acción) para los paquetes que coincidan
 - Extensible mediante módulos que se cargan con la opción `-m`
- **Objetivo** es la acción a realizar. Las acciones principales son:
 - **DROP:** descartar (borrar el paquete, como si nunca hubiera llegado → no da pistas a posibles atacantes)
 - **ACCEPT:** dejar pasar el filtro
 - **REJECT:** rechazar → envía mensaje de error
 - **LOG:** deja constancia de la llegada del mensaje en un archivo de log

La tabla filter

- Tabla por defecto con tres cadenas predefinidas:
 - **INPUT** para paquetes destinados a la máquina local
 - **OUTPUT** para paquetes generados localmente
 - **FORWARD** para paquetes encaminados a través esta máquina



Reglas en iptables

- Cada regla está definida por:
 - Un patrón a cumplir
 - Una acción a realizar (objetivo) para el paquete que encaja en el patrón
- Si el paquete no cumple ninguna regla se aplica política por defecto
- Las reglas se pueden agrupar en cadenas
 - Se irán evaluando todas las reglas hasta que se cumple algún patrón definido en alguna de ellas

Sintaxis de las reglas en Iptables

Principales criterios de filtrado

Opción/Ejemplo	Significado
-P INPUT	Establece política por defecto a la entrada
-A INPUT -A OUTPUT -A FORWARD	Añade regla a cadena de entrada Añade regla a cadena de salida Añade regla a cadena forward (solo en caso de routers)
-s 192.168.1.1 -d 140.10.15.1	Filtrado por dirección IP origen Filtrado por dirección IP destino
-p tcp -p udp -p icmp	Filtrado de paquetes TCP Filtrado de paquetes UDP Filtrado de paquetes ICMP
--sport 3000 --dport 80 --icmp-type 8	Filtrado por nº de puerto origen (solo para TCP o UDP) Filtrado por nº de puerto destino (solo para TCP o UDP) Filtrado por código del paquete ICMP (solo para ICMP)
-i eth0 -o eth1	Filtrado por interfaz de red de entrada Filtrado por interfaz de red de salida

Sintaxis de las reglas en Iptables

Filtrado por estado de la conexión

Opción	Significado
-m state --state NEW	Filtrado de paquetes correspondientes a conexiones nuevas (el primer paquete visto en una conexión)
-m state --state ESTABLISHED	Filtrado de paquetes correspondientes a conexiones ya establecidas
-m state --state RELATED	Filtrado de paquetes relacionados con otras conexiones existentes (Ej. conexión de datos FTP, o paquetes ICMP)
-m state --state INVALID	Filtrado de paquetes que no pertenecen a ninguno de los estados anteriores

Acciones

Opción	Significado
-j ACCEPT	El paquete es aceptado
-j DROP	El paquete es rechazado

Ejemplo reglas de filtrado

- Borrar reglas de la tabla FILTER (otra tabla con `-t`)

```
# iptables -F (# iptables -t nat -F)
```

- Establecer una política, por defecto, de descartar paquetes reenviados

```
# iptables -P FORWARD DROP
```

- Aceptar paquetes de Internet (eth1) hacia un servidor *web*

```
# iptables -A FORWARD -i eth1 -p tcp --dport http -d <web-server> -j ACCEPT
```

- Aceptar paquetes de la red interna (eth0) hacia Internet

```
# iptables -A FORWARD -i eth0 -j ACCEPT
```

Ejemplo reglas de filtrado

- Aceptar paquetes de Internet de conexiones establecidas

```
# iptables -A FORWARD -i eth1 -m state --state ESTABLISHED -j ACCEPT
```

- Mostrar reglas de la tabla FILTER (otra tabla con -t)

```
# iptables -L -v
```

- Introducir una regla en la posición 2

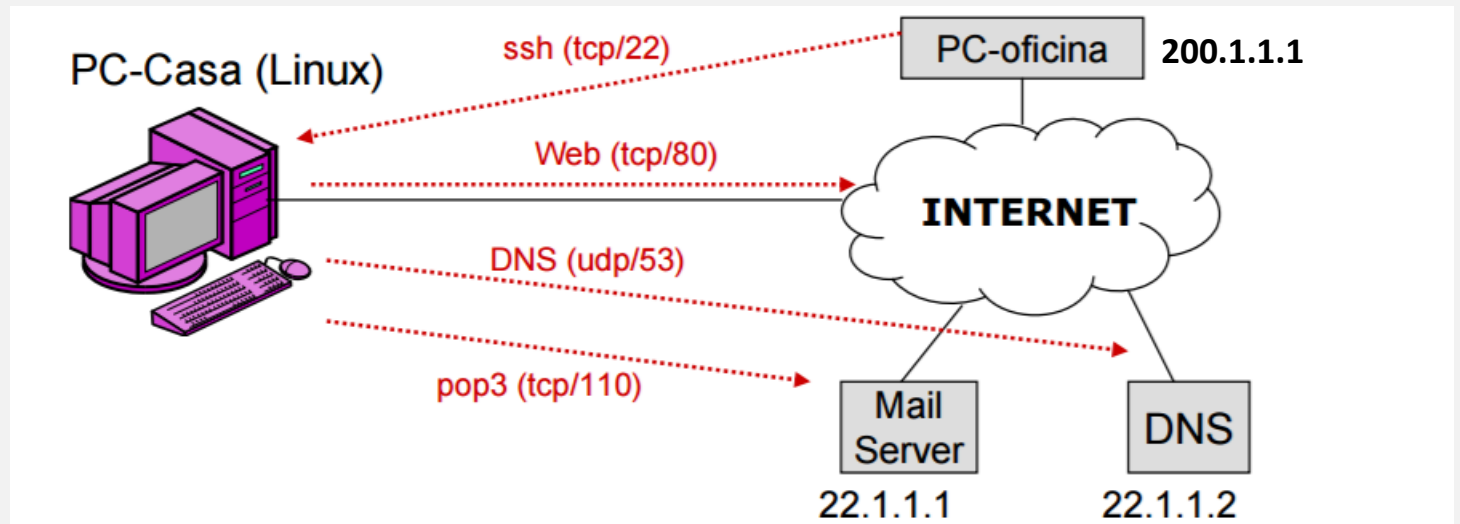
```
# iptables -I FORWARD 2 -i eth0 -j ACCEPT
```

- Borrar regla de la posición 2

```
# iptables -D FORWARD 2
```

Ejemplo reglas de filtrado

- Conexiones entrantes permitidas
 - Servicio SSH desde PC-oficina (200.1.1.1)
- Conexiones salientes permitidas
 - Servicio web a cualquier destino
 - Servicio pop3 a servidor de correo (22.1.1.1)
 - Servicio DNS a servidor DNS (22.1.1.2)
- Permitimos conexiones relacionadas o establecidas
- Resto conexiones: RECHAZADAS



Ejemplo tomado del profesor Rafael Moreno

Ejemplo reglas de filtrado

```
# Establecemos política por defecto para cadenas INPUT, OUTPUT y FORWARD
iptables -P INPUT DROP
iptables -P OUTPUT DROP
iptables -P FORWARD DROP
# Dejamos entrar o salir cualquier paquete correspondiente a
# conexiones establecidas o relacionadas
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# Permitimos conexiones entrantes SSH (tcp/22) desde pc-oficina
iptables -A INPUT -s 200.1.1.1 -p tcp --dport 22 -m state \
    --state NEW -j ACCEPT
# Permitimos conexiones web salientes (tcp/80) a cualquier destino
iptables -A OUTPUT -p tcp --dport 80 -m state --state NEW -j ACCEPT
# Permitimos conexiones pop3 salientes (tcp/110) con servidor de correo
iptables -A OUTPUT -d 22.1.1.1 -p tcp --dport 110 -m state \
    --state NEW -j ACCEPT
# Permitimos conexiones DNS salientes (udp/53) con servidor DNS
iptables -A OUTPUT -d 22.1.1.2 -p udp --dport 53 -m state \
    --state NEW -j ACCEPT
```

Cortafuego capa de aplicación

Cortafuegos de capa de aplicación

- **Deep Packet Inspection (DPI) firewalls:** capaces de inspeccionar el contenido del campo datos del paquete
 - Pueden bloquear nombres de dominio
 - Pueden acceder al **campo datos** de los paquetes lo que le permite:
 - Tomar decisiones en función del contenido y de la aplicación
 - ✓ Descartar un correo electrónico con un adjunto no permitido por la política de seguridad
 - ✓ No permitir páginas web con ActiveX o Java
 - Detectar comandos sospechosos de posibles ataques DoS o *buffer overflow*
 - ✓ Comandos repetidos
 - ✓ Un comando que no va precedido de otro que lo debe preceder
- **WAF (Web Application Firewall):** protege de múltiples ataques al servidor de aplicaciones web
 - Examina cada petición enviada al servidor antes de que llegue a la aplicación para asegurarse de que cumple con las reglas del firewall

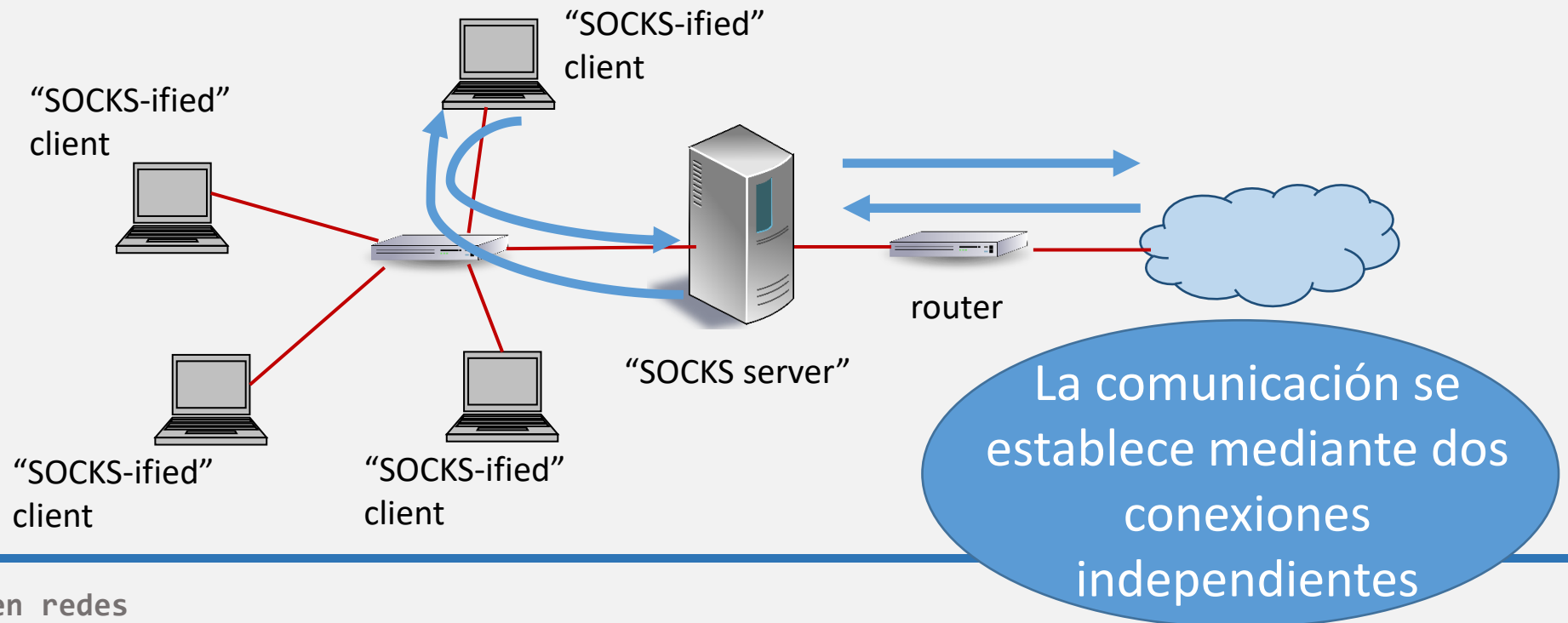
Proxy

Proxy

- Actúan como **intermediario** entre dos partes que se quieren comunicar
 - Nunca se permite la comunicación directa entre ellas
 - El trafico **entra por un puerto** del proxy **y se reenvía por otro puerto**
- Intercepta los paquetes y los inspecciona antes de enviarlos al destino
- Las **IPs internas** permanecen **ocultas** al exterior
 - Un host externo que se quiera conectar a la red local solo conoce la IP del proxy.
- Hace una copia de toda la información que transmite
- Según en qué nivel de la pila TCP/IP actúa existen dos tipos:
 - Proxy a nivel de circuito
 - Proxy de aplicación

Proxy a nivel de circuito

- Actúa como **intermediario a nivel de transporte**
 - No permite una conexión TCP de extremo a extremo
- Determina qué conexiones se permiten y crea un **canal seguro** entre las partes
- Proporciona seguridad a una **amplia variedad de protocolos** de aplicación
 - No hace inspección profunda** de los paquetes
- Un ejemplo de implementación es **SOCKS**

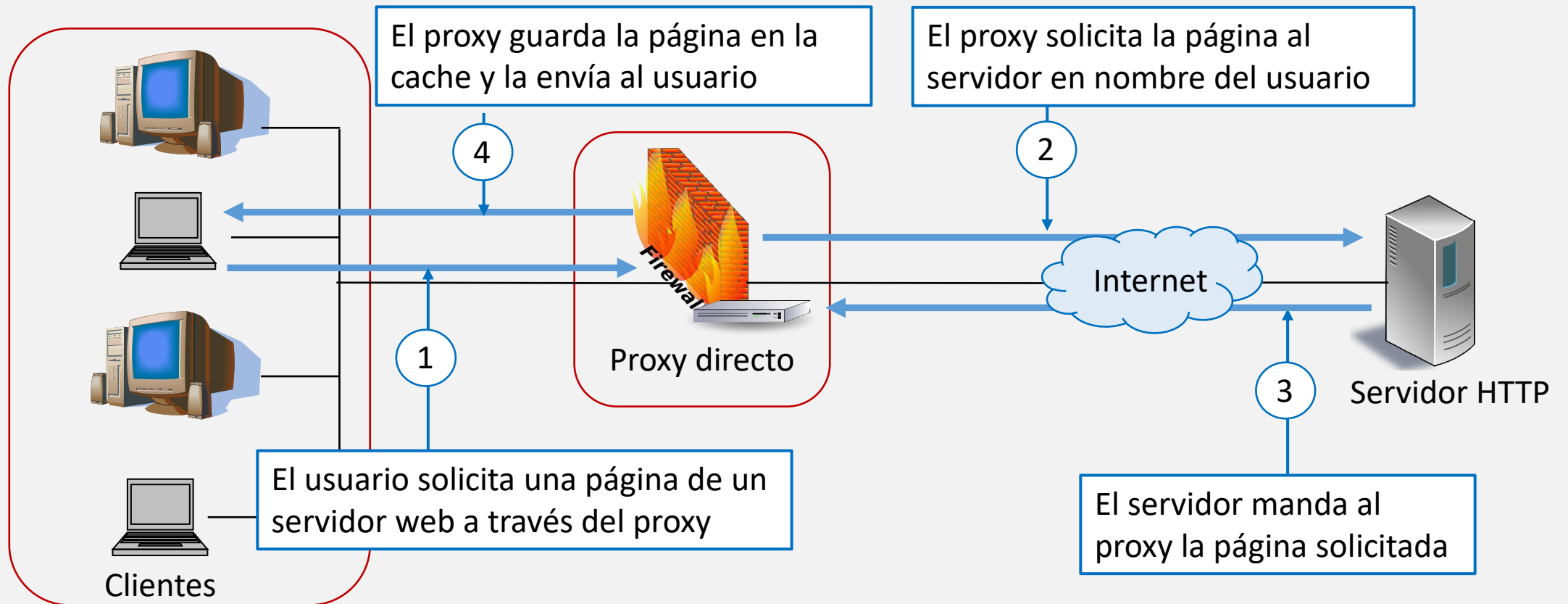


Proxy de aplicación

- Actúa como **intermediario a nivel de aplicación**
- Realiza el **filtrado** en función del **contenido del paquete** (campo datos)
- Conoce **comandos específicos** de un **protocolo**
 - ✓ Puede permitir paquetes que contengan el comando FTP GET y rechazar aquellos con FTP PUT
- Se requiere un **proxy diferente por protocolo** o servicio (FTP, HTTP, SMTP, ...)
- Mantiene información detallada de **auditoría**
- Introduce **sobrecarga** debido a la profunda inspección de los paquetes
 - ✓ No recomendado en aplicaciones de alto ancho de banda o en tiempo real
- Puede requerir **autenticación** de usuarios
- Los más comunes son los **web proxies**

Proxy directo

- **Forward proxy:** Protege a un cliente o grupo de clientes, normalmente en la misma red (ej. Squid, Tinyproxy)



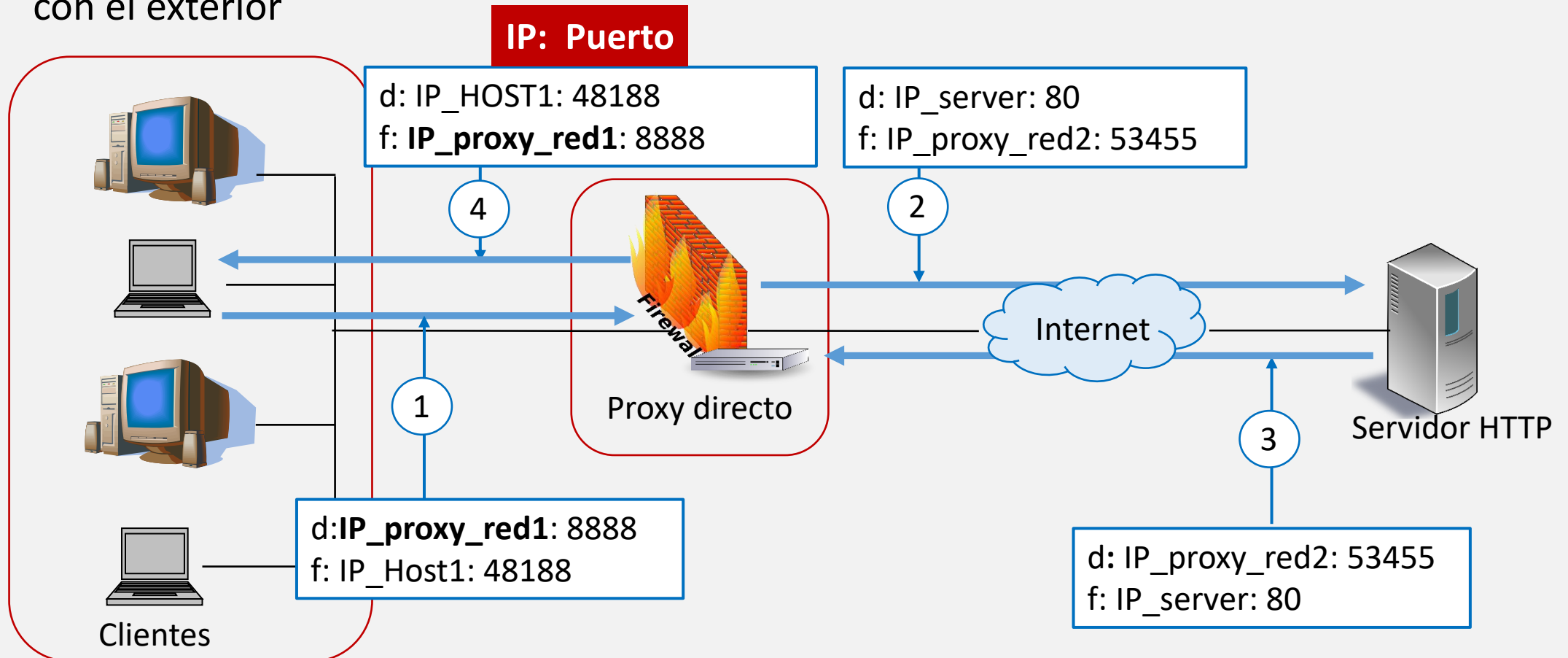
Proxy directo

- Para que funcione es necesario redirigir todo el tráfico a través del proxy:
 - **Los puertos y las direcciones IP** de la red interna no se pueden ver desde el exterior
 - Se puede implementar de dos formas:
 - Realizando la **configuración en el cliente**
 - **Configurando el proxy para funcionar en modo NAT => proxy transparente**

Proxy directo

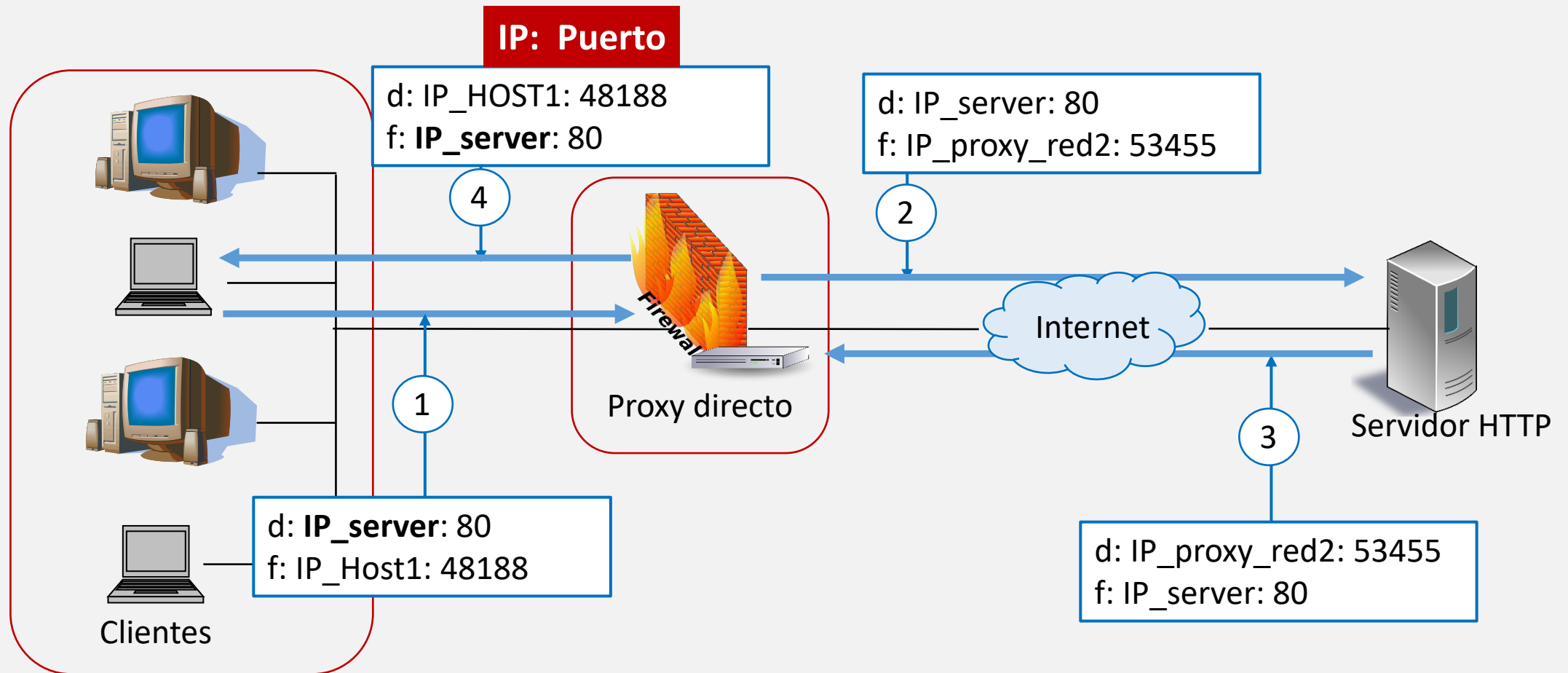
■ Configuración en el cliente

- El cliente se configura para poner como destino el puerto del proxy cuando se comunica con el exterior



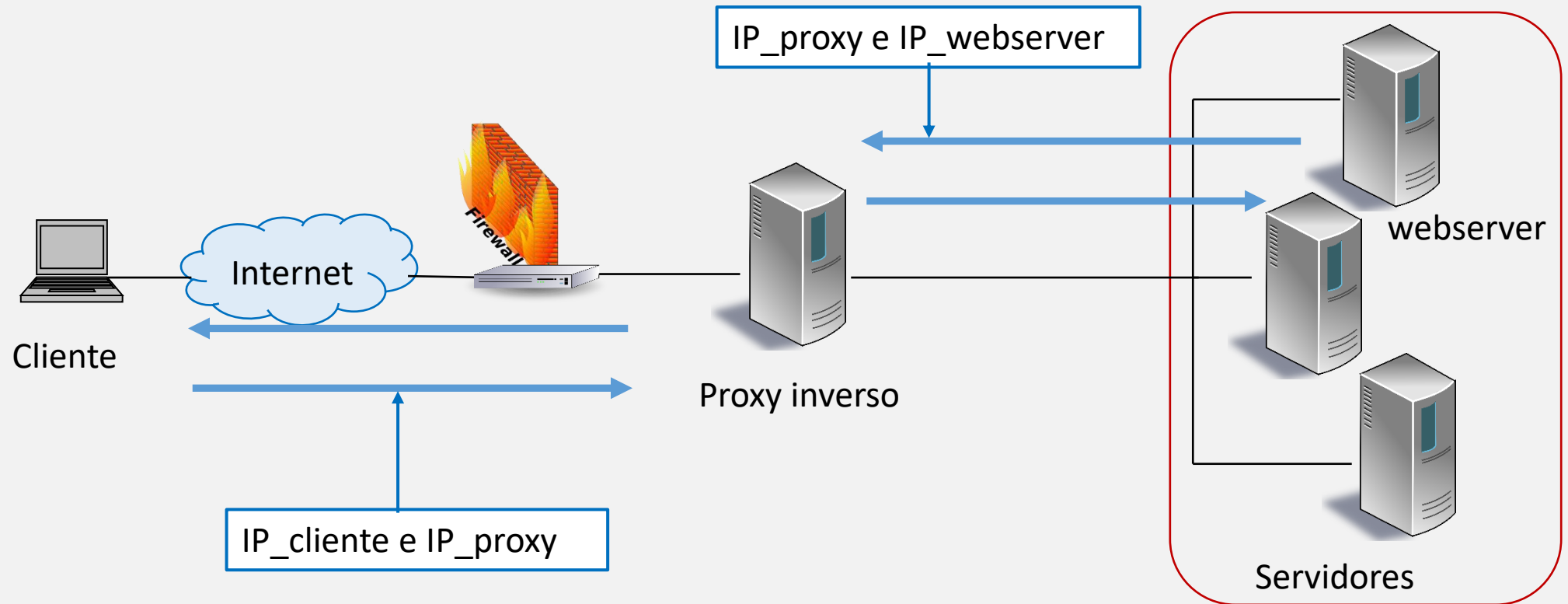
Proxy directo

- Configurando el proxy para funcionar en modo NAT => proxy transparente
 - El cliente no sabe que hay un proxy y cree que se comunica directamente con el exterior



Proxy inverso

- *Reverse proxy (proxy server)*: es un servidor ordinario que protege a los servidores reales de entradas maliciosas (ej. ModSecurity)



Evolución de los cortafuegos

Nueva generación

(NGFW, Next-generation firewall)

✓ Incorpora un IPS basado en firmas

- Puede buscar por indicadores específicos de un ataque
- Algunos pueden compartir la firma de un nuevo ataque en la nube, los demás cortafuegos del mismo vendedor podrán prevenir ese ataque
- Caros, podrían no ser viables para redes de tamaño pequeño o medio

2ª generación

- ✓ Son capaces de **monitorizar el tráfico** para detectar ataques basados en software (*buffer overflows, injections, malware, ...*)

1ª generación

- ✓ Solo podía monitorizar el tráfico de red y evitaba ataques basados en red (escaneo, DoS, ...)

Los cortafuegos siempre deberían...

- Fundamental: Utilizar una política por defecto de bloqueo
- Rechazar paquetes con direcciones IP no válidas (p.e. 192.168.1.1) y el tráfico entrante con destino una dirección de difusión
- Rechazar paquetes salientes con una IP origen que no pertenece a la red
 - Se evita así que los sistemas de la red actúen como agentes (*zombies*) en un ataque DoS distribuido
- Rechazar paquetes entrantes con una IP origen perteneciente a la red local
 - *IP spoofing* que se usa para evadir el cortafuegos
- Reensamblar fragmentos para poder examinar el paquete completo y evitar ataques de fragmentación
- Rechazar los paquetes que llegan con la opción encaminamiento de origen activada

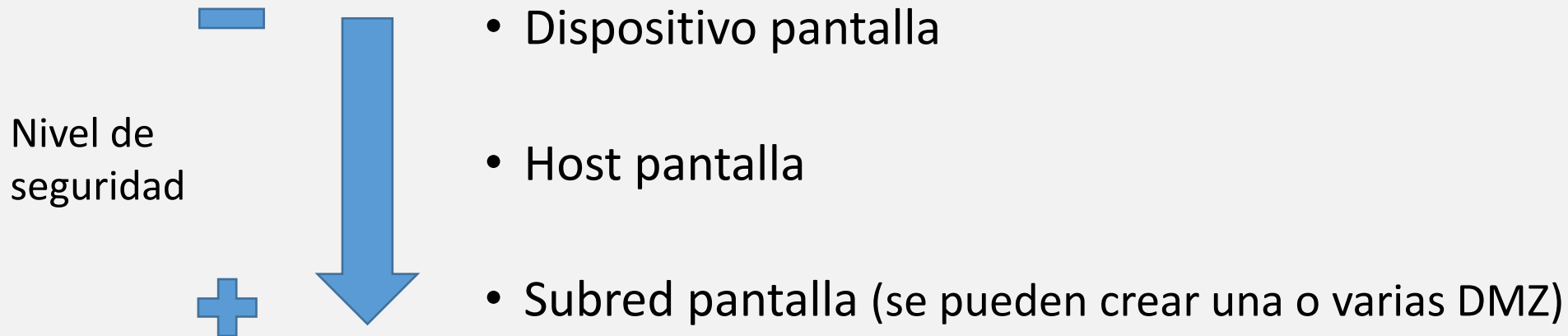
Arquitecturas de cortafuegos

Arquitectura de cortafuegos

- Para que un **cortafuegos** pueda cumplir con su misión de **proteger los activos** de una organización, esta debe asegurarse de:
 - Haber seleccionado **la tecnología de cortafuegos adecuada**
 - Situar el cortafuegos en el **lugar apropiado**
- Los cortafuegos se pueden situar en **distintos sitios en función** de lo que se **pretenda proteger**
 - En la **parte más externa** de la red local: **protege** esta red frente al **tráfico** proveniente del **exterior**
 - Pueden utilizarse para crear una red perimetral (DMZ, zona demilitarizada)
 - **Dentro** de la **red local**: para crear **distintos segmentos** dentro de una misma red y controlar el acceso entre ellos
- Casi todas las organizaciones tienen las mismas necesidades
 - Los cortafuegos se suelen colocar en los mismos sitios

Arquitectura de cortafuegos

- En función de dónde se sitúa el o los cortafuegos dentro de la arquitectura de la red, hablamos de:



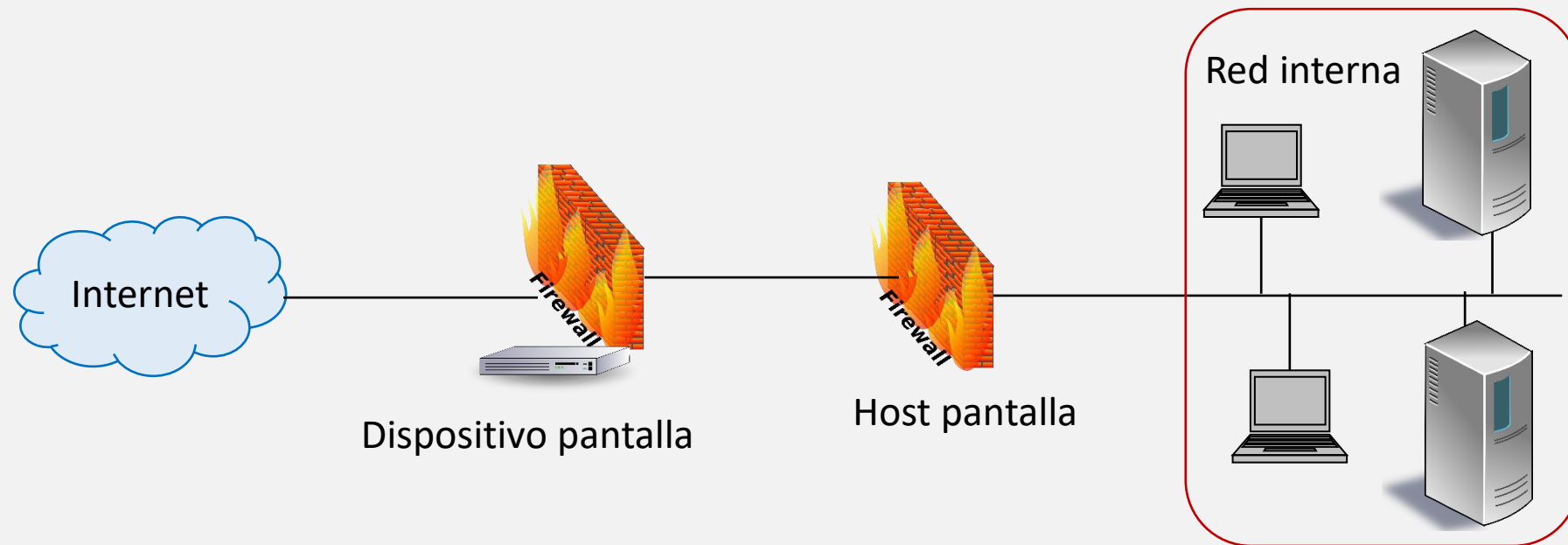
Dispositivo pantalla

- Dispositivo de dos o más interfaces al que se le ha instalado un software de cortafuegos
- Conecta la red interna directamente con el exterior (router+cortafuegos)
- Problema: Todo el tráfico desde el exterior hacia la red pasa por este dispositivo
 - Si es atacado, la red queda comprometida



Host pantalla

- Dispositivo situado entre un router y la red interna
- **Dos niveles de filtrado:**
 - Primer nivel: en el **router**, **filtrado de paquetes**
 - Segundo nivel: en el **host pantalla**, suele tener un **cortafuegos** de **capa de aplicación**



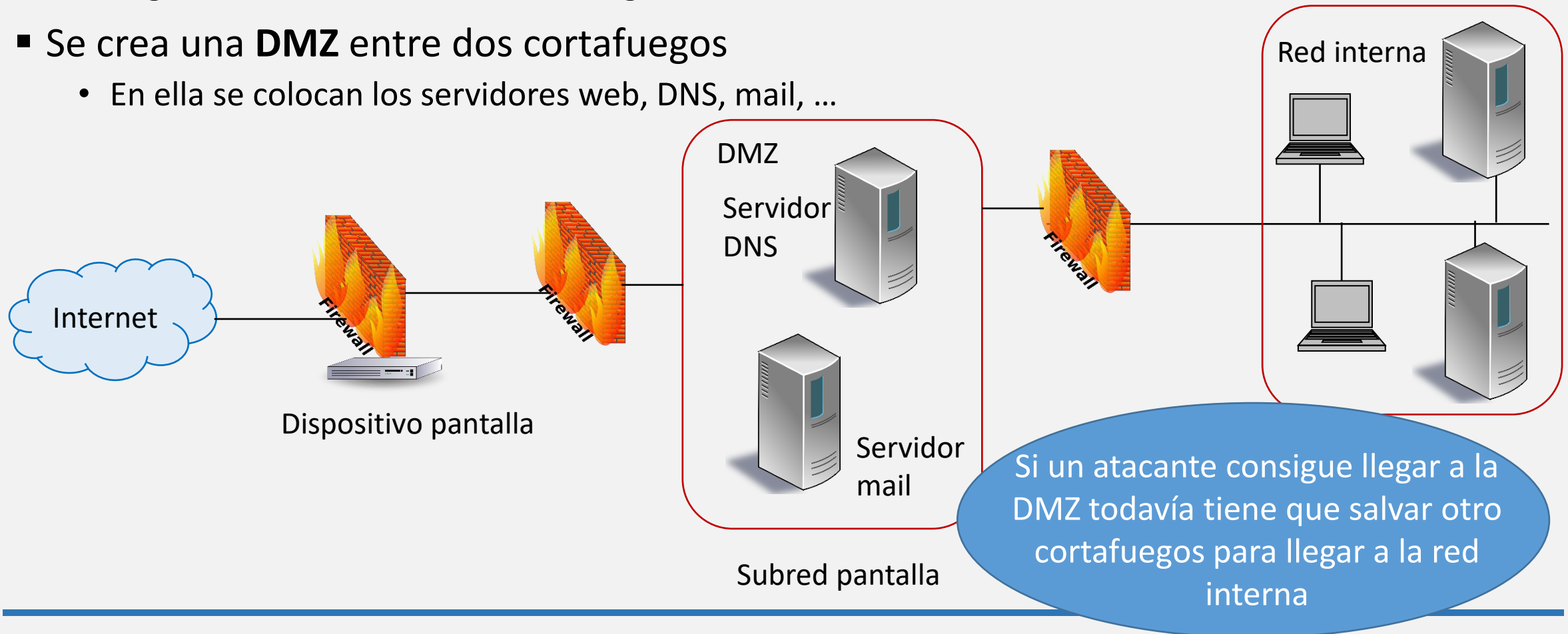
Subred pantalla

■ Tres niveles de filtrado

- Los cortafuegos deben ser diferentes para conseguir mayor seguridad
- Reglas más estrictas en los cortafuegos más internos

■ Se crea una **DMZ** entre dos cortafuegos

- En ella se colocan los servidores web, DNS, mail, ...



Dos subredes pantalla

- Aún más niveles de filtrado

